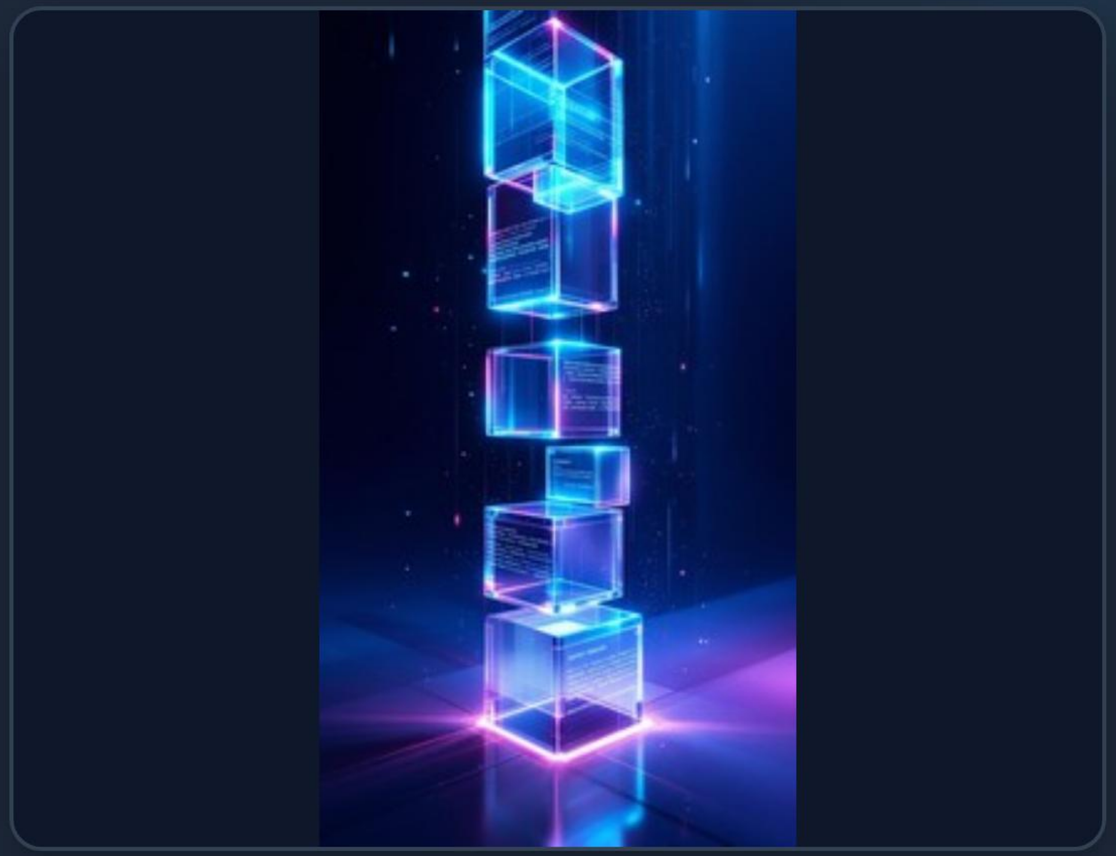
Struktur Data: Stack & Queue

Memahami konsep, operasi dasar, dan implementasi dari dua struktur data fundamental dalam ilmu komputer.

Bagian 1: Memahami Stack

Konsep LIFO (Last In, First Out)

- → Stack beroperasi sepenuhnya berdasarkan prinsip **LIFO**.
- → Elemen yang terakhir dimasukkan (Last In) akan menjadi elemen pertama yang diambil (First Out).
- → Struktur ini dapat dibayangkan seperti **tumpukan piring**: piring yang ditaruh di atas adalah yang pertama kali harus diambil.
- → Hanya memiliki satu akses, yaitu bagian puncak (top) dari struktur.



Operasi Dasar pada Stack

Push: Menambahkan elemen baru ke posisi paling puncak (top) dari stack.



Pop: Menghapus sekaligus mengembalikan nilai elemen yang ada di puncak stack.



Peek / Top: Membaca elemen yang berada di puncak stack tanpa menghapusnya.



IsEmpty: Fungsi pengecekan untuk mengetahui apakah stack sedang dalam keadaan kosong.

Latihan Soal: Operasi Stack

Simulasi Stack (Soal 1)

Sebuah Stack kosong diberikan operasi-operasi berikut secara berurutan:

```
1. Push(10)
2. Push(25)
3. Pop()
4. Push(40)
5. Push(50)
6. Pop()
```

Berapakah nilai yang berada pada bagian puncak (**Top**) dari stack saat ini, dan bagaimana isi elemennya dari bawah ke atas?

Jawaban: `Top = 40 | Isi: [10, 40]`

Simulasi Stack (Soal 2)

Terdapat operasi Stack di bawah ini. Tentukan nilai kembalian dari operasi **Pop** yang terakhir!

1. `Push(5)`
2. `Push(15)`
3. `Push(30)`
4. `Pop()`
5. `Push(45)`
6. `Pop()`
7. `Pop()`

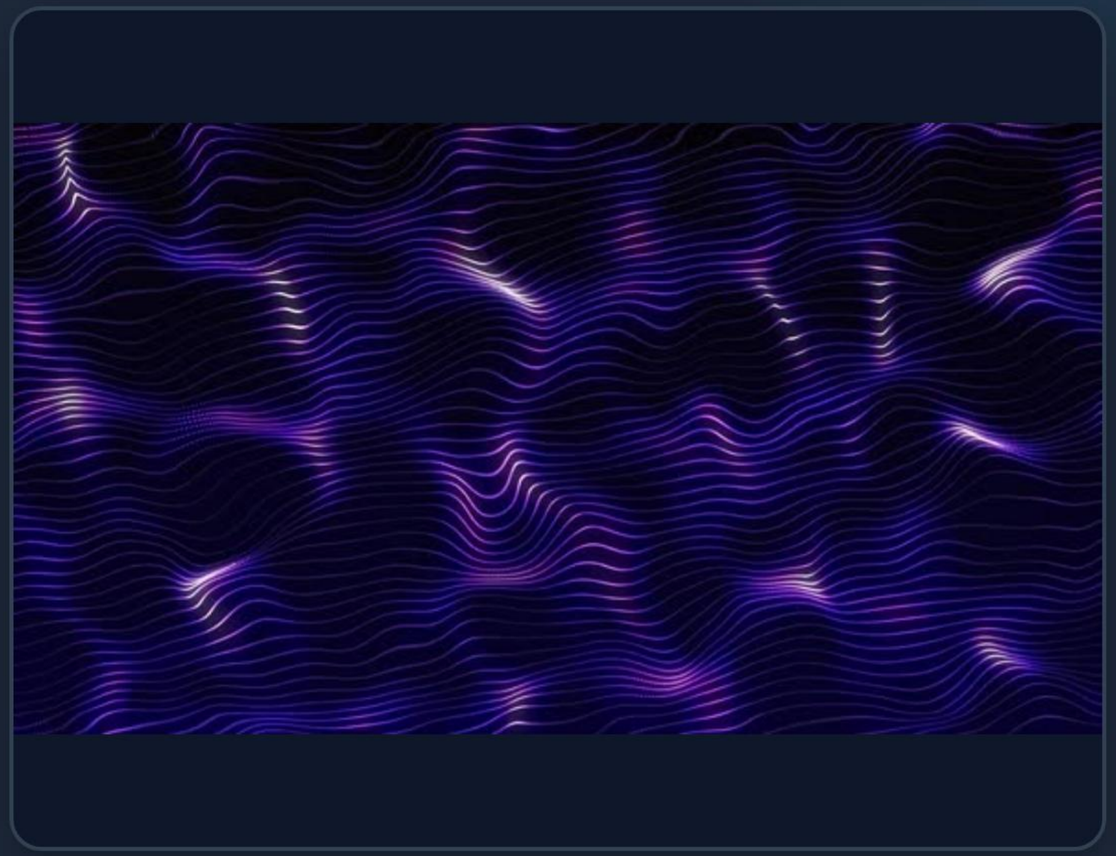
Nilai kembalian Pop() terakhir (Langkah ke-7): **15**

*Penjelasan: Setelah langkah 5, stack berisi [5, 15, 45]. Langkah 6 mengeluarkan 45. Langkah 7 mengeluarkan 15.

Bagian 2: Memahami Queue

Konsep FIFO (First In, First Out)

- → Queue beroperasi berdasarkan prinsip **FIFO** yang sangat teratur.
- → Elemen yang pertama kali masuk (First In) adalah elemen pertama yang akan dikeluarkan atau dilayani (First Out).
- → Sama persis seperti **antrean di kasir**: orang yang datang lebih awal akan dilayani lebih dulu.
- → Penambahan dilakukan dari belakang (Rear), sedangkan pengambilan dilakukan dari depan (Front).



Operasi Dasar pada Queue

- **Enqueue:** Menyisipkan elemen baru ke bagian paling belakang (rear / tail) antrean.
- **Dequeue:** Menghapus dan mengembalikan elemen yang berada di bagian paling depan (front / head).
- **Front / Peek:** Melihat elemen pertama di antrean tanpa mengubah struktur atau menghapusnya.
- **IsEmpty:** Fungsi krusial untuk mengecek apakah antrean sedang kosong atau berisi.

Latihan Soal: Operasi Queue

Simulasi Queue (Soal 1)

Sebuah Queue kosong (kapasitas tidak terbatas) diberikan operasi berikut:

```
1. Enqueue(12)
2. Enqueue(24)
3. Enqueue(36)
4. Dequeue()
5. Enqueue(48)
```

Berapakah elemen yang berada pada bagian **Front** (Depan) dan **Rear** (Belakang) dari queue saat ini?

Jawaban (Front ... Rear): `Front = 24 | Rear = 48`

*Isi Queue: [24, 36, 48]

Simulasi Queue (Soal 2)

Lacak keluaran dari operasi-operasi berikut. Urutkan angka yang dihasilkan (dikembalikan) oleh setiap operasi **Dequeue**!

1. `Enqueue(7)`
2. `Enqueue(14)`
3. `Dequeue()`
4. `Enqueue(21)`
5. `Dequeue()`
6. `Enqueue(28)`
7. `Dequeue()`

Urutan nilai yang di-Dequeue: `7, 14, 21`

*Tersisa angka 28 di dalam Queue.

Perbandingan Fundamental

Parameter Utama	Struktur Stack	Struktur Queue
Prinsip Kerja	LIFO (Last In First Out)	FIFO (First In First Out)
Titik Akses	Satu ujung saja (Top)	Dua ujung berbeda (Front & Rear)
Penambahan Data	Push (di Top)	Enqueue (di Rear/Belakang)
Penghapusan Data	Pop (dari Top)	Dequeue (dari Front/Depan)
Contoh Analogi	Tumpukan Kotak / Piring	Antrean Gerbang Tiket

Sesi Tanya Jawab

Apakah ada materi yang ingin didiskusikan lebih lanjut?

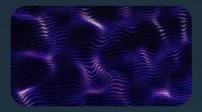
admin.strukturdata@universitas.edu

Image Sources



https://images.stockcake.com/public/9/0/2/902765b7-792e-4159-b1cb-299935e47263_medium/futuristic-glowing-cubes-stockcake.jpg

Source: stockcake.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/04ce50aa-b8ea-4960-a73f-0e6411a37425/video_preview/video_preview_0000.jpg?w=1200&h=630&cf_fit=crop&q=85&format=jpeg&s=866b618d2e8c1d21b5c699e49ada18edf65924785f0f1691fe19fb3f847f3825

Source: elements.envato.com